College Code : 9607

College Name : Immanuel Arasar JJ College Of Engineering

Department :CSE

NM-Id :aut960723104021

Roll No :15

Date :12-09-2025

Completed The Project Named As Phase: Sharmili R.S, Jaya krishnan P, sakthi A, Saranya S, kevin.S

Technology Project Name : single page application

Submited By ;

Sharmili R.S

Enhancements & Deployment

1.Additional features

User Experience (UX) Enhancements

Validation feedback in real time (display error messages during typing rather than after submission).Progress indicators (step progress bars for multi-step forms or loading spinners).Tooltips and Hints (inline messages or help icons that describe input fields).Toggle between Dark and Light Mode for ease of use.

🔷 **Functional Features**

Save as Draft (keep partially completed data in a database or local storage).Suggestions and auto-fill (e.g., city, country auto-complete using APIs).Reset the

form to quickly remove all entered values.Data import/export (CSV/JSON for submitted stored forms).

🔷 Data & Security

•client-side encryption prior to storing private information.

•validation against APIs and Regex (e.g., phone number verification, email check).

•Integration of Captcha (to stop bots, use Google reCAPTCHA or hCaptcha).

🔷 Analytics & Monitoring

•Error logging, which records unsuccessful submissions.

•Form Usage Analytics (most skipped fields, time spent in each field, and drop-offs).

🔷 Integration Features

email alerts when a form is successfully submitted.

Webhook Support (provides third-party apps with submitted data).

Database Sync (extend to backend DB if only local state is being used at the moment).

🔷 Accessibility & Compliance

WCAG-compliant inputs (ARIA labels, keyboard navigation).

Support for multiple languages (basic i18n).

Notice of GDPR/Privacy Compliance.

2. UI/UX improvements

🔷UI Improvements

- Consistent Layout & Spacing: For a neat structure, use a grid system and equal padding/margins.

•Contrast & Colour Scheme → Select a contemporary colour scheme that includes both light and accent hues. For readability, make sure the contrast is good.

•Typography Hierarchy → Various font sizes and weights for labels, input fields, headings, and errors.

•Visual Cues & Icons → Include subtle icons for the success/error states and fields (password, email).

•Micro-animations → Fluid changes upon button clicks, focus, hover, and error highlights.

•Fluid input sizes and a stacked layout on small screens are examples of responsive design's optimisation for desktop, tablet, and mobile devices.

🔷 **UX Improvements**

•Inline Validation → Display error messages not only when submitting but also while typing.

•Unambiguous Error Message → Say "Password must be 8+ characters with a number" rather than the standard "Invalid input."

•Success Factors → Valid fields are indicated with green checkmarks or faint highlights.

•Focus Management After submitting, automatically move the cursor to the first invalid field.

•Progressive Disclosure Only display additional fields when necessary (for example, "Company Name" if the user chooses "Business").

•Accessible Form Controls → Larger clickable areas, labels tied to inputs, ARIA roles.

•Keyboard Navigation → Ensure Tab/Enter works naturally across inputs.

•Auto-Save Drafts → Prevent data loss if the page refreshes.

•Confirmation Screen → Show a “review before submit” or “thank you” screen with submitted details.

🔷 **Nice-to-Have Enhancements**

•The theme toggle (light/dark mode) → Updates the app's appearance and ease of use.

•Tooltip/Aided Text → Give complex fields inline hints.

•Show users their progress if the form is multi-step.

•Allow users to reverse unintentional form resets with the Undo Option.

3. API Enhancement

1.Functional Improvements

- Add new endpoints, such as /notifications and /user/preferences.

•Extend existing responses with optional fields; these must stay backward compatible.

•Add improved filters, sorting, and search features.

•Support bulk or batch requests to lessen multiple round trips.

Performance Enhancements

• Introduce pagination and lazy loading.

•Enable caching with HTTP cache headers, Redis, and CDN.

•Use compression, such as gzip and Brotli, to reduce payload size.

•Adopt GraphQL or Backend-for-Frontend (BFF) to avoid over-fetching or under-fetching.

•Optimize database queries and add indexes.

Security Enhancements

•Introduce pagination and lazy loading.

•Enable caching with HTTP cache headers, Redis, and CDN.

•Use compression, such as gzip and Brotli, to reduce payload size.

•Adopt GraphQL or Backend-for-Frontend (BFF) to avoid over-fetching or under-fetching.

•Optimize database queries and add indexes.

Observability Enhancements

•Add structured logging and correlation IDs.

•Monitor latency, throughput, and error rates.

•Integrate tracing, such as OpenTelemetry.

•Expose health and status endpoints for monitoring.

4.Performance & Security Checks

1. **Performance Checks.**

1. To make sure the app can handle the expected loads, simulate a lot of user traffic.
2. JMeter, Locust, and k6 are some of the tools.
3. Testing for Stress and Spikes

4. To find out where the system breaks down, push it past its normal limits.
5. Check for sudden spikes, like going from 100 to 10,000 users in a few seconds.
6. Response Time and Latency
7. Make sure that API calls respond within the SLA you set (for example, <200ms).
8. Keep an eye on how long it takes to run DB queries.
9. Throughput and Scalability
10. Count the number of transactions per second (TPS).
11. Check how well it scales horizontally and vertically.
12. Profiling Memory and CPU
13. Find memory leaks and too much CPU use.
14. Tools: New Relic, Prometheus + Grafana, and VisualVM.
15. Performance of Caching and Databases
16. Make sure that queries are optimised (joins and indexes).
17. Check the caching strategy (Redis, in-memory cache).
18. Performance on mobile and networks
19. Try it out on different devices and networks, like Wi-Fi and 3G/4G/5G.
20. Make it work better when you're not online or have a slow internet connection.

2.Security Checks.

1. Authorisation and Authentication
2. Implement JWT, OAuth2, and strong password policies.
3. RBAC stands for role-based access control.
4. Encryption of Data
5. Encrypt data while it's in transit (TLS 1.2/1.3).
6. Encrypt critical information while it's at rest using AES-256.
7. Protect API secrets and keys (Vault, environment variables).

8.Validation and Sanitisation of Input

9.Avoid CSRF, XSS, and SQL Injection.

10.Verify user input on the client and server sides.

11.Security of Sessions and Tokens

12.Make use of short-lived tokens that have refresh mechanisms.

13.Use HttpOnly and SameSite flags to secure cookies.

After logging out, invalidate tokens.

5.Testing of Enhancements

1. Define the Enhancement

Enhancements could mean:

•Genetic modification, for example, engineered receptor-binding proteins.

•Formulation, such as improved stability, encapsulation, or lyophilization.

•Delivery method, including aerosol, oral, topical, or IV.

•Spectrum broadening, like phage cocktails or engineered host range.

•Safety or resistance-reduction mechanisms.

2. In-Vitro Testing

•Host range assays: Spot tests, plaque assays against expanded bacterial strains.

•Efficiency of plating (EOP): Measure infectivity compared to wild type.

•Bacterial kill curves: Monitor OD_{600} decline to assess lytic activity.

•Biofilm disruption assays: Use crystal violet staining and viable cell counts.

•Resistance monitoring: Pass bacteria with phage over multiple generations to see how quickly resistance develops.

•Stability tests: Test heat, pH, UV, and storage stability.

3. In-Vivo / Ex-Vivo Testing

•Animal models (mice, Galleria mellonella, zebrafish): infection model with or without phage.

•Tissue culture: Check the cytotoxicity of the phage preparation on mammalian cells.

•Biodistribution and clearance: qPCR or plaque counts in tissues over time.

•Immunogenicity: Use ELISA to measure the antibody response to phage.

4.Safety & Quality Testing

•Endotoxin levels, LAL assay for purified phage preps.

•Absence of virulence or toxin genes, whole genome sequencing (WGS).

•No lysogenic activity, screen for integrases if lytic therapy is intended.

•Sterility checks, confirm no bacterial contaminants.

5. Comparative Performance

- Compare enhanced phage and wild type side by side:
- Killing efficiency.
- Stability, shelf life.
- Host range breadth.
- Resistance suppression.

6.Regulatory & Translational Testing

•GMP manufacturing feasibility, scalability of the improvement.

•Toxicology studies that follow FDA and EMA guidelines.

•Controlled clinical trial design, if therapeutic: safety in Phase I and efficacy in Phase II and III.

6.Deployment (Netlify, Vercel, or Cloud Platform)

1. Netlify Deployment for SPA

•setup

•Push your SPA code (React, Angular,vue etc.) to GitHub, GitLab, or Bitbucket.

•In the Netlify dashboard, select New Site from Git.

•Connect your repo and set the build command and the publish directory (build/ for React,dist / for value).

•SPA routing fix

create a _redirect file in "public/"

/* /index.html 200

This makes sure that client-side routes, like /dashboard, don't result in a 404 error.

2. Vercel Deployment for SPA

•setup

• Install Vercel CLI using npm i -g vercel or connect your GitHub repo.

•Run vercel in your project root.

•Vercel automatically detects the framework, like React, Next.js, Vue, or Angular.

•Set the build command to npm run build and specify the output directory, such as build/ or dist/.

SPA routing fix:

Add a vercel.json file:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

3. Cloud Platforms (AWS / GCP / Azure)

•AWS S3+ cloudFront

•Build your SPA locally by running npm run build.

•Upload the build/ folder to an S3 bucket.

•Enable static website hosting in S3.

•Add CloudFront CDN for global caching and HTTPS.

•For SPA routing, go to CloudFront error pages and redirect all 404s to /index.html.

•**Google Cloud Storage + Firebase Hosting**

• Firebase Hosting

•npm install -g firebase-tools

•firebase init hosting

•Select SPA. Then, deploy using firebase deploy.

•Add "rewrites": [{ "source": "**", "destination": "/index.html" }] in firebase.json.

•**Azure Static Web Apps**

•Connect the repository in the Azure portal.

•Set up the build by specifying app_location and output_location.

•SPA routing is managed through routes.json.